Hospital Management System

CSE110 (OBJECT ORIENTED PROGRAMMING)

MINI PROJECT

NAME	ID	CONTRIBUTION
❖ Rifa azad	2022-1-60-265	
❖ Tasnova Ahmed Mila		
❖ Moumita MIm		

Course Instructor: Mostofa Kamal Rasel

"Hospital Management System"

Introduction:

❑ *Background:*

➢ Reasons behind selecting this project:

- Hospital System is pretty much familiar to all of us; thus it will be easier to implement and think various ideas to include in the project.
- Hospital Management System has a hierarchy which can be use to build ***"is-a relationship"*** using ***interface***.
- Because of the hierarchy, it will be easier to demonstrate different syntax of **OOP**(**O**bject **O**riented **P**rogramming). Almost every concept of **JAVA** can be applied with little bit of twist and turn.

➢ This project is important because:

- It will be a good practice to implement everything we learned in this course: **CSE110**
- This project is a reasonable way to judge our skill in OOP and a medium to learn more and expand **JAVA** skills.

❑ ***Problem Statements:***

While doing the project we face many technical problems such as logical errors, syntax errors, also understanding and finding solution to these were a great effort. Few Problems we faced are listed below:

- **Appending Data** using ObjectOutputStream-

  It is not possible to append data in ObjectOutputStream as Java serialization does not support ***"appending".*** After some exploration, found out that Typically with stream based classes (especially higher order streams like **OOS**), need to rewrite the whole file, anytime you update it. But we have already written data using **OOS** and wanted to use this stream to write which became a time-consuming and difficult task.

  But it was possible to handle this problem; which will be discussed following on.

- For ***"searching"***, will have a user define data and search through the stream and show that object's all data. But using every method (*contains(), equals(), compareTo(), equalsIgnoreCase(), ...)* and applying different ways of code*(ArraList search, List, ListITeration, searching after writing as Text I/O),* the output of this particular code was not what it should be.

No matter how the approach is, program was not able to find specific data(user input) from the file.

- Part of the project ***Doctors*** ***(a child class of Staff class)*** class was having problem to print it's given values to its objects. The output was showing all the default values that were set by the class ***constructor.*** Solution for this will be discussed in later on.

❑ *Objectives:*

Regardless of problems we faced while doing the project, we tried our best to solve those and show a definite output. Some was solved by effort and exploring on web by finding reason and how to solve using different approach. Such as:

- In **OOS,** ***"appending"*** data was not possible but by creating a ***subclass*** of **OOS** and ***overriding*** its method solved of the problem of ***"appending.*** Creating own **MyObjectOutputStream** class, extending **ObjectOutputStream** (***inheritance***) and ***overriding*** the method : "*protected void writeStreamHeader()*" . In this new class, this method should do nothing.

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;

class MyObjectOutputStream extends ObjectOutputStream {

    // Constructor1 of this class
    MyObjectOutputStream() throws IOException {
        super();
    }

    // Constructor2 of this class
    MyObjectOutputStream(OutputStream o) throws IOException{
        super(o);
    }

    1 usage
    public void writeStreamHeader() throws IOException{
        return;
    }
}
```

Now, to write object for the first time need to use predefined **OOS** but after creating once need to use own **MyObjectOutputStream**. Then it will be possible to append data in a already exists file without losing old data.

But this was still a problem as ObjectInputStream was not able to read the newly written data but always reading up to pre-written data. So to solve this came to a different approach and re-written the whole code which is (in pseudo code):

```
User define input;
Arraylist create;
File create to get the path;
Declaring ObjectOutputStream, ObjectInputStream;//but not creating them

try{
    if(file exists){
        objIn = new ObjectInputStream(new FileInputStream(filePath));
        store the read values to arraylist.
        Close the stream;
    }
  else{//means file does not exist
        objOut = new ObjectOutputStream(new FileOutputStream(filePath));
        create Patient class Object s and add them to Arraylist;
        objOut.writeObject(patients);
        close stream and program exist;
  }
//appending new data ; will come here after if block
        objOut = new ObjectOutputStream(new FileOutputStream(filePath));
        //takes user inputs as many they want then store them in ArrayList and then write to the file
// after writing done closes the stream
}catch(Exception){
}
//by using ObjectInputstream opens the file and prints the data in console.
```

Following this way ***Doctors*** and ***Nurses*** classes were written as well; as this way was prove to be more efficient.

- Even after trying many approach, the problem faced while doing search was not solved. From my perspective, I believe that maybe InputStream was not

able to match given data with its inside data because of unusual spacing. Ways that were used to solve this given below(even though few codes were lost while trying to solve the problem):

```java
        try{
            if(filePath.isFile()) {
                objIn = new ObjectInputStream(new FileInputStream(filePath));
                patientRegister = (ArrayList<Patient>) objIn.readObject();
                objIn.close();
                System.out.println(patientRegister.toString().replace( target: "[",  replacement: "").replace( target: ",",  replacement: "\n").replace( target: "]",  replaceme
                System.out.print("Enter the name you want to search: ");
                String search = input.next();
                PrintWriter pw = new PrintWriter( fileName: "Searched Patients.txt");
                pw.print(patientRegister);
                String[] info = new String[7];
                Scanner find = new Scanner( source: "Searched Patients.txt");
                while(find.hasNextLine()){
                    String data = find.next();
                    info = data.split( regex: ",");
                    String name = info[0];
                    String name2 = info[1];
                    String date = info[2];
                    String age = info[3];
                    String illness = info[4];
                    String cabin = info[5];
                    String serial = info[6];
                    if(name.equalsIgnoreCase(search)){
                        System.out.println("found");
                    }
                    else System.out.println("NOO");
                }
```

```java
    Scanner input = new Scanner(System.in);
    ArrayList<Patient> patientRegister = new ArrayList<>();
    PrintWriter pw = new PrintWriter( fileName: "Searched Patients.txt");
    pw.print(patientRegister);

    System.out.print("Enter the name you want to search: ");
    String search = input.next();
    ListIterator list;
    list = patientRegister.listIterator();
    while(list.hasNext()){
        Patient p = (Patient) list.next();
        if(p.getName() == search){
            System.out.println("Found");
        }
        else
            System.out.println("Not");
    }
}
```

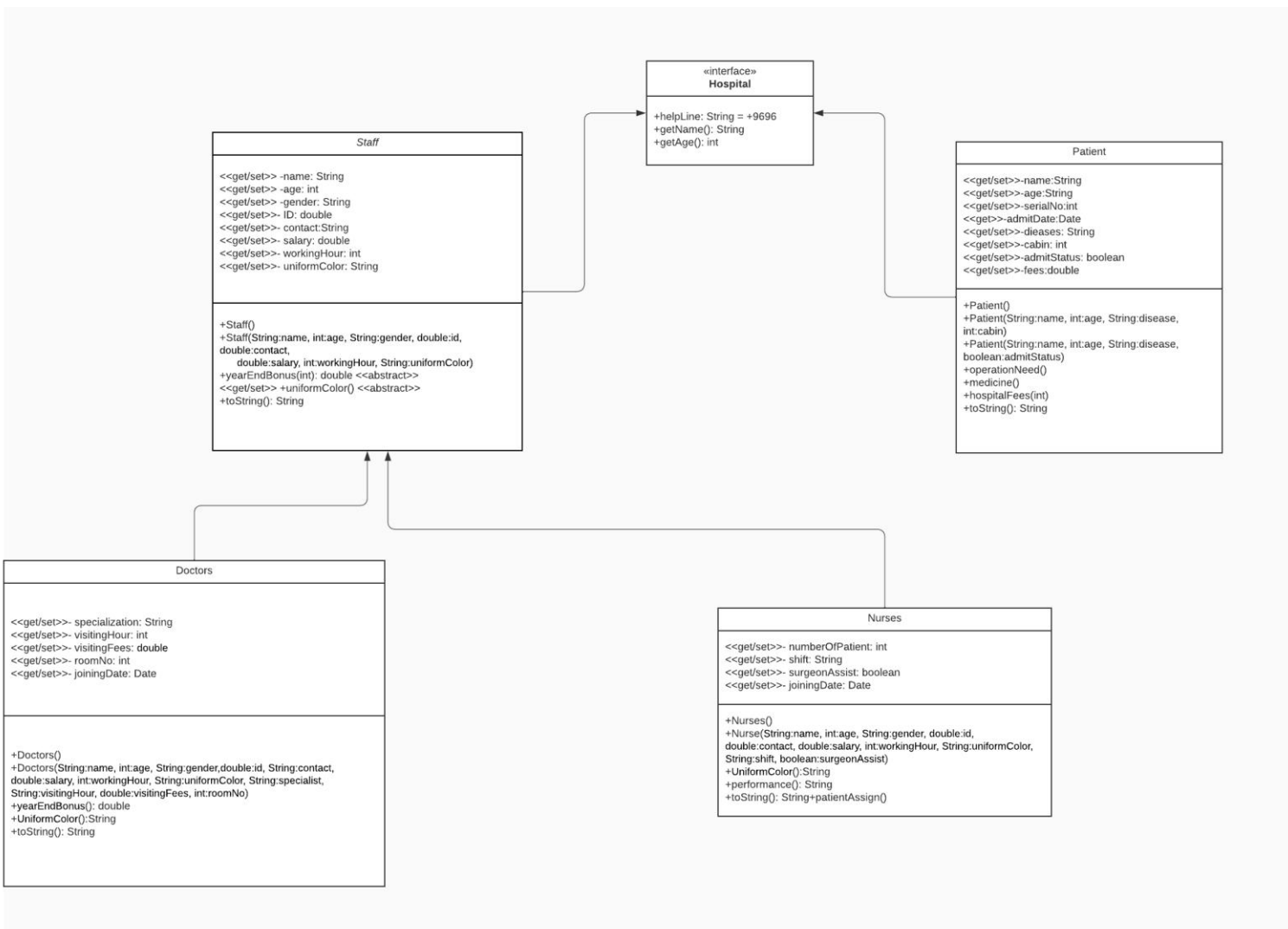
```java
public static void main(String[] args) throws FileNotFoundException {
    Scanner input = new Scanner(System.in);
    ArrayList<Patient> patientRegister = new ArrayList<>();
    PrintWriter pw = new PrintWriter( fileName: "Searched Patients.txt");
            pw.print(patientRegister);

    System.out.print("Enter the name you want to search: ");
    String search = input.next();
            Scanner find = new Scanner( source: "Searched Patients.txt");
            while(find.hasNextLine()){
                String data = find.nextLine();
                if(data.equalsIgnoreCase(search)){
                    System.out.println("found");
                }
                else
                    System.out.println("NOO");
            }

            pw.close();

}
```

Above are given two other approaches ; before trying with ***ListIterator List*** with ***ArrayList*** was tried as it helps in search but due to some problem it did not work.

- The nest problem faced was default value output of ***Doctors***; it was solved by ***serializing Staff*** class.

Planned Methodology: